Test and ship

Databricks pipeline engineer

A9 Testing

A9-L1 pytest basics, and a SparkSession fixture shared by every test

One session-scoped fixture, and tests with known answers that fail for the bugs you care about.

```python
def test_weighted_share_uses_weights() -> None:
    assert weighted_share([1, 0], [3.0, 1.0]) == 0.75
```

```python
@pytest.fixture(scope="session")
def spark() -> Iterator[SparkSession]:
    session = SparkSession.builder.master("local[2]").getOrCreate()
    yield session
    session.stop()
```

```python
@pytest.mark.parametrize(("flags", "weights", "expected"), [([1], [2.0], 1.0), ([0, 0], [1.0, 1.0], 0.0)])
def test_share(flags, weights, expected): ...
```

What does pytest run?	Functions named test_* in files named test_*.py. Plain assert statements.
SparkSession fixture scope	scope="session": start Spark once per test run.
yield in a fixture	Hands the value to tests. Code after yield is the teardown.
What makes a test worth having?	It fails for the bug you fear: known answers that separate right from wrong, plus edge cases.

A9-L2 A fast local SparkSession and readable test data

A tuned local session makes tests fast enough to run on every change. Builders with explicit schemas make them short and exact.

```python
builder = (
    SparkSession.builder.master("local[2]")
    .config("spark.sql.shuffle.partitions", "2")
    .config("spark.ui.enabled", "false")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .config("spark.sql.sources.default", "delta")
)
spark = configure_spark_with_delta_pip(builder).getOrCreate()
```

```python
def respondent(**overrides: object) -> dict[str, object]:
    row = {"respondent_id": "R1", "wave_id": "W1", "final_weight": 1.0, "in_scope": True}
    row.update(overrides)
    return row
```

Local test session settings	master("local[2]"), shuffle partitions 2, UI off, Delta configured with configure_spark_with_delta_pip.
Default spark.sql.shuffle.partitions	200: far too many for test data.
Why explicit schemas in test data?	Inference fails on all-None columns and can pick different types from production.

Test data builder	A function with valid defaults. Tests override only the fields they are about, and typos raise.

A9-L3 Compare DataFrames properly, and test that MERGE is idempotent

One helper compares DataFrames the right way, and one small test proves a MERGE survives a retry and a repeated key.

```
from pyspark.testing import assertDataFrameEqual, assertSchemaEqual

assertDataFrameEqual(actual, expected)
```

Compare two DataFrames in a test	pyspark.testing.assertDataFrameEqual(actual, expected). Ignores order, tolerates float rounding, checks schema.
When should row order matter?	Only for ranked output. Pass checkRowOrder=True.
Idempotence test for a MERGE	Load, snapshot with collect(), load again, assertDataFrameEqual(table, snapshot).
Why collect() for the snapshot?	A DataFrame is lazy and would re-read the table after the second run.

A10 Deployment

A10-L1 Git folders, one branch per change, and no secrets in the repository

Code flows from a branch, through review and CI, to production. Secrets flow from a secret store at run time, never through Git.

```
api_key = dbutils.secrets.get(scope="research", key="supplier_api_key")
```

Where do developers work in Databricks Git folders?	Their own clone under /Workspace/Users/<email>/, on their own branch.
Who updates production Git folders?	Only automation, after a pull request merges into the deployment branch.
Read a secret in a notebook	dbutils.secrets.get(scope="...", key="..."). Output shows [REDACTED].
A token was committed, then removed. Is it safe?	No. Git history keeps it. Rotate the token.

A10-L2 Declarative Automation Bundles and parameterised jobs

One reviewed file describes the job for every environment. The target decides where it writes.

```
bundle:
  name: research_pipelines

variables:
  catalog:
    description: Catalog the pipelines write to.

resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
```

```
        - name: catalog
          default: ${var.catalog}
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py

targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: production
    variables:
      catalog: research_prod
```

```
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev nat_silver_pipeline
```

What are Declarative Automation Bundles?	Jobs and their settings as YAML in Git (formerly Databricks Asset Bundles). Same CLI: databricks bundle.
Targets and variables	Targets are environments. Variables hold what differs, such as the catalog, set per target.
What does mode: development do?	Prefixes resources with [dev <user>] and pauses schedules and triggers.
Read a job parameter in a notebook	dbutils.widgets.get("name"). Validate it before using it.

A10-L3 CI: lint, type-check, test and validate on every pull request

Every pull request is linted, typed, tested and validated. Only a merge to main deploys, as a service principal, with nothing secret in the file. The CLI version shown is an example: pin the version you tested.

```
env:
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
```

Four CI checks for a pipeline repository	ruff (lint and format), mypy, pytest with local Spark, databricks bundle validate.
Who deploys to production?	CI, after a merge to main, as a service principal.
OAuth M2M environment variables	DATABRICKS_HOST, DATABRICKS_CLIENT_ID, DATABRICKS_CLIENT_SECRET from the CI secret store.
Two ways CI lies	continue-on-error: true, and command || true. Both turn failures green.